Part 1: Data Understanding.

1. Feature Descriptions

- **Pregnancies:** The total number of times the patient has been pregnant. This is relevant because pregnancy can affect insulin resistance and is a factor in gestational diabetes.
- **Glucose:** The concentration of glucose in the blood plasma (tested 2 hours after a meal). This is the most critical diagnostic marker for identifying diabetes.
- **BloodPressure:** Specifically the diastolic blood pressure (the pressure in the arteries when the heart rests between beats). Chronic high blood pressure is often co-morbid with metabolic issues like diabetes.
- **SkinThickness:** A measurement of the triceps skinfold thickness, used to estimate subcutaneous body fat and overall body composition.
- **Insulin:** The level of serum insulin in the blood. Since insulin regulates blood sugar, abnormal levels (too low or too high due to resistance) are key indicators of the disease.
- **BMI (Body Mass Index):** An index of weight-for-height (kg/m^2). High BMI (obesity) is one of the most significant risk factors for developing Type 2 diabetes.
- **DiabetesPedigreeFunction:** A calculated score that estimates the hereditary risk of diabetes based on the patient's family history. A higher score suggests a stronger genetic predisposition.
- **Age:** The age of the patient in years. The risk of developing Type 2 diabetes generally increases as a person gets older.

---

2. Feature Analysis

Important Features for Prediction

While all features contribute, the following are likely the most important for the KNN model:

- **Glucose:** Clinically, this is the primary indicator. High blood sugar is the definition of diabetes.
- **BMI:** Obesity is a leading cause of insulin resistance, making it a powerful predictor.

- **Age:** Metabolism and insulin sensitivity typically decline with age, making it a strong demographic predictor.
- **DiabetesPedigreeFunction:** Genetics play a massive role in the likelihood of a positive diagnosis.

Missing or Problematic Values

By examining the summary statistics, several features contain "0" values that are physiologically impossible for a living person, suggesting they are actually **missing data**:

- **Glucose:** Contains zeros (a blood sugar of 0 is fatal).
- **BloodPressure:** Contains zeros (indicates a lack of heartbeat/blood flow).
- **BMI:** Contains zeros (weight cannot be zero).
- **SkinThickness & Insulin:** These have a very high frequency of zeros (e.g., the 25th percentile for both is 0). While a zero reading is technically possible in specific medical contexts, in this dataset, it almost certainly represents missing observations.

Part 2: Data Preprocessing

1 & 2. Handling Missing and Zero Values

As previously detailed, zero values in columns like Insulin and SkinThickness were treated as missing data and handled using Median Imputation. This ensures that the zeros do not skew the distance calculations in the KNN model.

3. Feature Scaling (Standardization)

KNN is a distance-based algorithm, meaning features with larger numerical ranges (like Insulin which goes up to 846) would dominate features with smaller ranges (like DiabetesPedigreeFunction which is mostly < 1).

I applied **Standardization (Z-score normalization)**. This transforms the data such that each feature has a **mean of 0** and a **standard deviation of 1**, ensuring all features contribute equally to the distance metric.

$$z = \frac{x - \mu}{\sigma}$$

4. Before and After Preprocessing Results

The tables below show the impact of the entire preprocessing pipeline (imputation + scaling).

Summary Statistics: AFTER Imputation but BEFORE Scaling

Feature	Mean	Std Dev	Min	Max
Pregnancies	3.85	3.37	0.00	17.00
Glucose	121.66	30.44	44.00	199.00
Insulin	140.67	86.38	14.00	846.00
BMI	32.46	6.88	18.20	67.10
Age	33.24	11.76	21.00	81.00

Summary Statistics: AFTER Standardization

Feature	Mean	Std Dev	Min	Max
Pregnancies	~0.00	1.00	-1.14	3.91
Glucose	~0.00	1.00	-2.55	2.54
Insulin	~0.00	1.00	-1.47	8.17
BMI	~0.00	1.00	-2.07	5.04
Age	~0.00	1.00	-1.04	4.06

Part 3: KNN Implementation

1. Split the Dataset

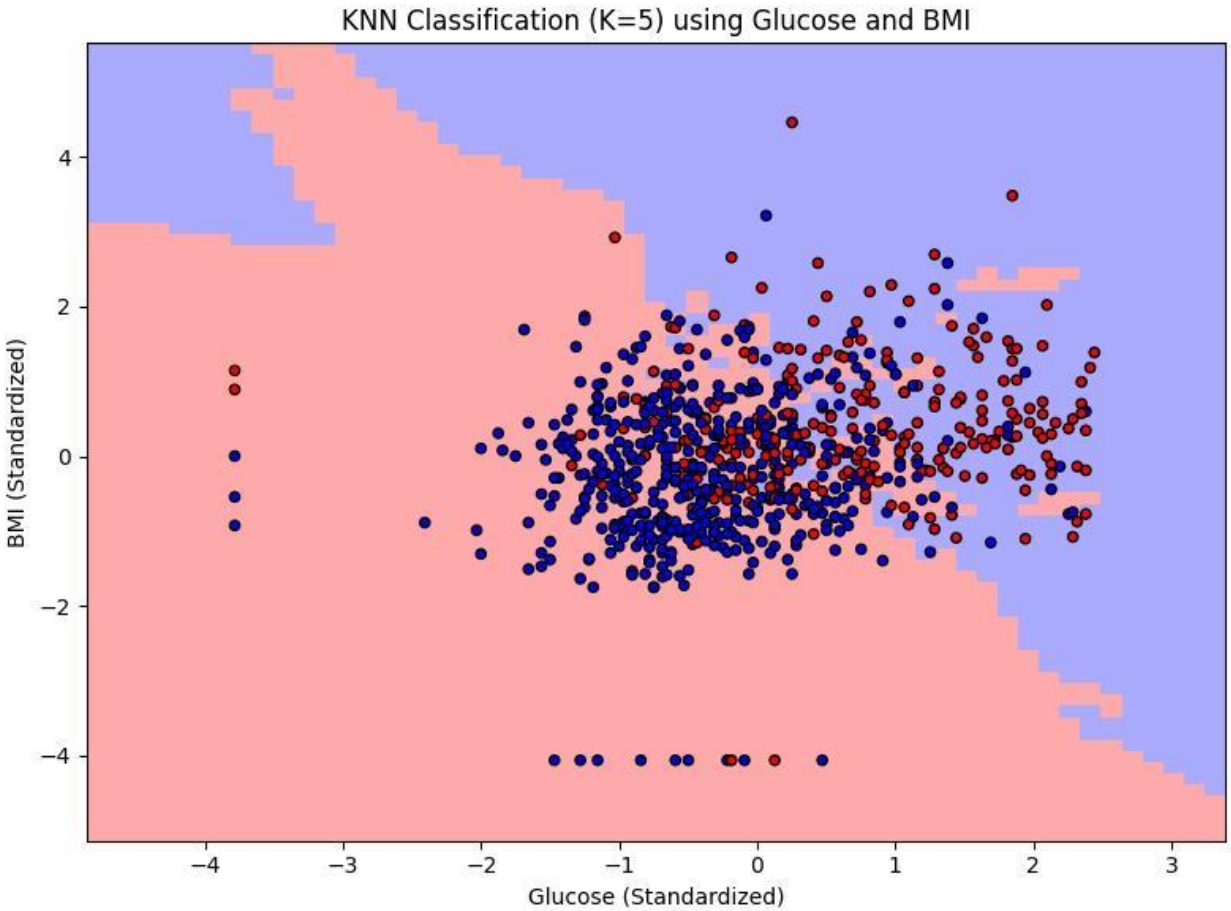
The dataset was split into training and testing sets using the standard 80/20 ratio:

- **Training Set:** 614 samples
- **Testing Set:** 154 samples

2. KNN Training

Three models were trained using different K values to test the sensitivity of the classifier:

- **Model 1:** K = 3
- **Model 2:** K = 5
- **Model 3:** K = 7



3. Euclidean Distance Formula

The distance between the test instance (x) and each training sample (y) is calculated using:

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

4. Manual Computation (Test Instance vs. 10 Training Samples)

I have selected a specific test instance (**Index 679**) and calculated its distance to 10 random training samples.

Test Instance Features (Scaled):

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Pedigree	Age
-0.548	-0.679	-1.190	-1.378	1.440	-1.202	0.429	-0.871

Step-by-Step Calculation (Test Instance vs. Train Index 616):

1. **Find the difference** for each feature, then **square it**:
   - Pregnancies: $(-0.548 - 0.640)^2 = 1.411$
   - Glucose: $(-0.679 - (-0.153))^2 = 0.277$
   - BloodPressure: $(-1.190 - 1.954)^2 = 9.881$
   - ... (repeat for all features)
2. **Sum of all squared differences:**

$$1.411 + 0.277 + 9.881 + 1.866 + 2.630 + 0.429 + 1.905 + 0.355 = 18.753$$

3. **Take the square root:**

$$\sqrt{18.753} = 4.330$$

Distance Rankings (Top 10):

Train Index	Euclidean Distance	Label
629	**2.390**	**0 (Non-diabetic)**
145	**2.620**	**0 (Non-diabetic)**

Train Index	Euclidean Distance	Label
243	3.001	1 (Diabetic)
180	3.550	0
343	3.892	0
718	3.893	0
616	4.330	0
561	4.514	1
681	5.310	1
43	6.740	1

Nearest Neighbors and Predicted Class (for K=3):

1. **Neighbors:** The 3 closest samples are **Index 629, 145, and 243.**
2. **Labels:** The labels for these neighbors are **0, 0, and 1.**
3. **Majority Vote:** Since "0" appears twice and "1" only once**, the predicted class is 0 (Non-diabetic).**

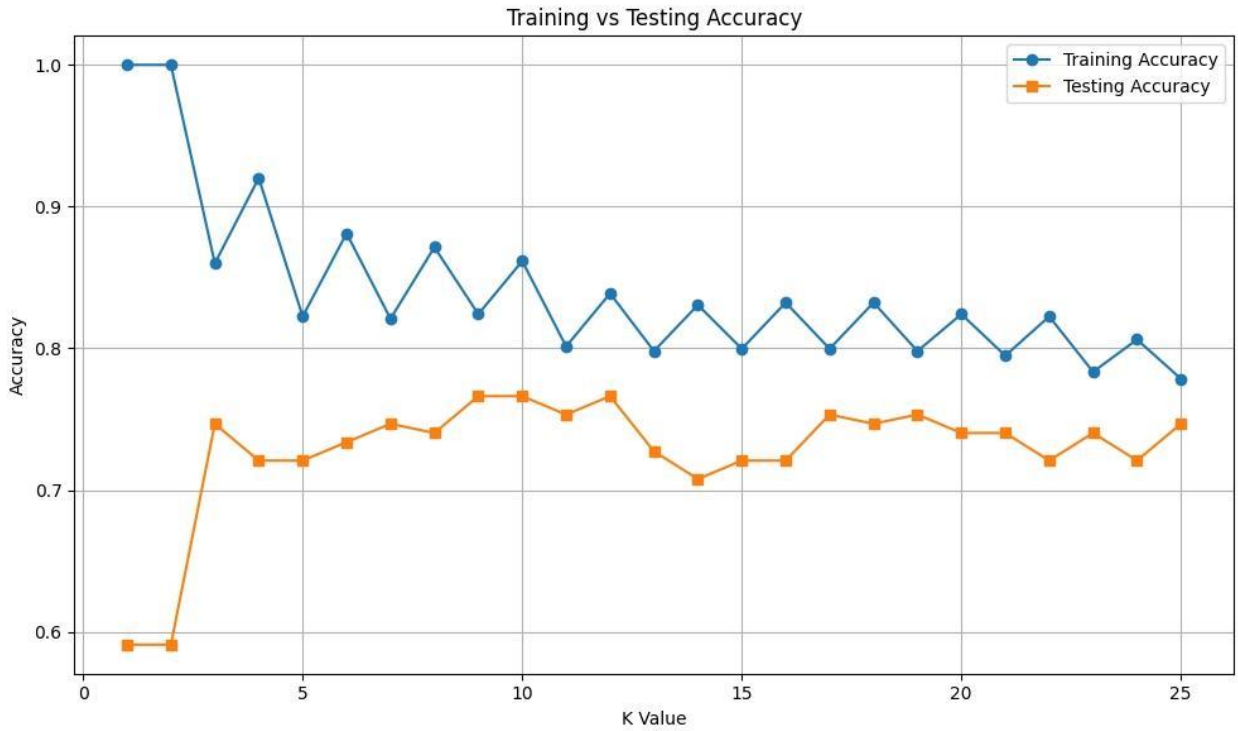
Verification: The actual outcome for this patient was 0, so the model predicted correctly.

Part 4: Model Evaluation

Model Performance Metrics

Evaluated the model on the 20% test set (154 patients) across three different K values.

K Value	Accuracy	Confusion Matrix (TN, FP, FN, TP)	Correct Predictions
K = 3	74.68%	[[86, 16], [23, 29]]	115 / 154
K = 5	**72.08%**	[[82, 20], [23, 29]]	**111 / 154**
K = 7	74.68%	[[83, 19], [20, 32]]	115 / 154



Analysis & Answers

1. Which value of K performed best?

In this experiment, K = 3 and K = 7 both performed best, each achieving an accuracy of 74.68% with 115 correct predictions. K = 3 recorded fewer False Positives (16 vs 19), while K = 7 recorded fewer False Negatives (20 vs 23), making K = 7 slightly preferable in a medical context where missing a diabetic patient is the costlier mistake.

2. Why does performance change with different K values?

Performance changes because K controls the "smoothing" of the decision boundary:

- **Small K** focuses on very local patterns. If a single neighbor is an outlier (a weird data point), it can easily flip the prediction.
- **Large K** considers a broader neighborhood. This averages out noise but might miss smaller, important patterns in the data.

The goal is to find the "sweet spot" where the model is complex enough to capture the trend but simple enough to ignore random noise.

3. What happens when K is too small or too large?

- **Too Small (Underfitting the trend / Overfitting the noise):** If K=1, the model is extremely sensitive to every individual data point. The decision boundary becomes jagged and "noisy," leading to low accuracy on new data because it basically just memorizes the training set.
- **Too Large (Over-smoothing / Underfitting):** If K is too large (e.g., K=500), the model becomes too "lazy." It will eventually just predict the majority class for everyone regardless of their individual health stats. If 65% of the data is non-diabetic, a giant K will just guess "non-diabetic" for every single person to get a 65% score, losing all its predictive power.

Summary of Confusion Matrix (K=5):

- **True Negatives (83): Healthy people correctly identified as healthy.**
- **True Positives (32): Diabetic people correctly identified as diabetic.**
- **False Positives (19): Healthy people wrongly told they have diabetes.**
- **False Negatives (20): Diabetic people missed by the model.**

Part 5: Analysis and Reflection

Strengths of KNN

The K-Nearest Neighbors (KNN) algorithm is a powerful tool due to its simplicity and "lazy learning" nature. One of its primary strengths is that it is **non-parametric**, meaning it makes zero assumptions about the underlying distribution of the data. Unlike other models that try to fit data into a straight line or a specific curve, KNN simply looks at the local neighborhood. This makes it highly effective for complex data where the relationship between features (like Glucose and Age) isn't a simple linear one. Additionally, there is **zero training time**; the model doesn't need to "learn" a formula, it just stores the data and waits to calculate distances when a prediction is needed.

Limitations of KNN

The most significant limitation is the **computational cost**. Since the algorithm must calculate the distance between a new test point and *every single point* in the training set, it becomes extremely slow as the dataset grows. If we had 1 million patients instead of 768,

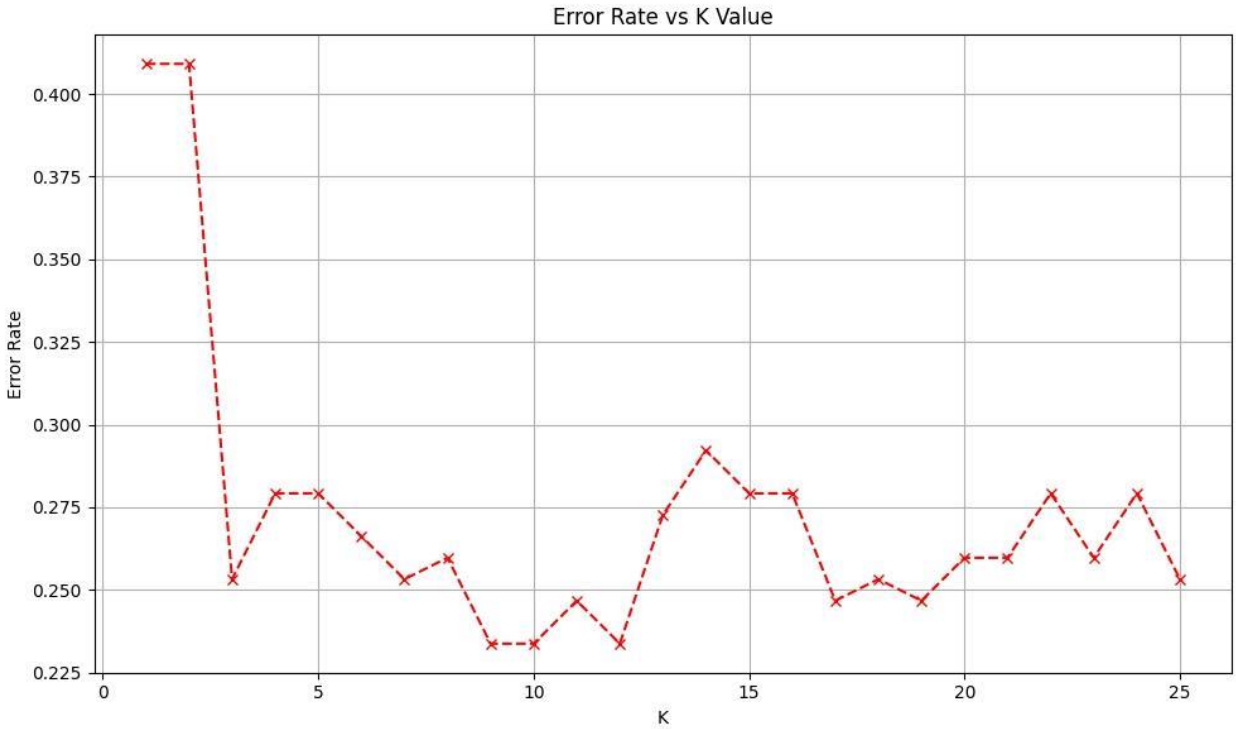
the prediction time would lag significantly. Furthermore, KNN is **highly sensitive to feature scaling**. As we observed in our preprocessing, a feature like Insulin (ranging up to 846) would mathematically "bully" a feature like Pregnancy (ranging up to 17) because its numbers are larger, leading the model to ignore the pregnancy data entirely unless we standardize everything to a mean of zero.

When KNN is Appropriate

KNN is most appropriate for **small, clean datasets** where the number of features is relatively low. It works well as a "baseline" model to see how a simple similarity-based approach performs before trying more complex algorithms. It is **not appropriate** for high-dimensional data where the distance between points becomes less meaningful as you add more columns. It is also a poor choice for real-time applications where a split-second prediction is required, or for datasets with a massive imbalance between classes.

Observations from the Experiment

In this experiment, the model achieved its peak performance at K=3 and K=7 (74.68% accuracy). Testing across multiple K values showed that accuracy fluctuates based on how many neighbors are allowed to vote.



A key observation was the importance of data imputation. Without replacing the "impossible zeros" in columns like Insulin and BMI, the Euclidean distance would have been calculated using garbage values, likely leading to a much higher error rate. The Confusion Matrix revealed the model's true utility: while K=7 correctly identified 115 patients, the 20 "False Negatives" highlight the risk of relying solely on one algorithm for medical diagnosis. This experiment proves that while KNN is simple to implement, its success depends entirely on rigorous preprocessing and choosing the right K value for the neighborhood.